



High-Performance Computer Architectures Practical Course

Introduction to Neural Networks

Prof. Dr. Ivan Kisel

Robin Lakos

Akhil Mithran

Oddharak Tyagi

May 13, 2025

There are many different architectures and types of neural networks...

- this is not a machine learning lecture, therefore we will only cover basics here
- only one of the simplest types of neural networks is roughly covered within this course
- every programming exercise is simplified to basic math (e.g. affine transformation), such that no deep ML knowledge is required

Then, why neural networks?

- many ongoing research topics
- used in many different disciplines (computer science, physics, medicine, economics, ...)
- exercises are hopefully more interesting with some modern topic as a context
- applications of HPC concepts are possible

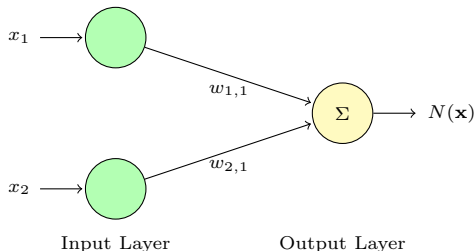
Perceptron: Historical Context

The simple perceptron was presented by Rosenblatt in 1957.

The main idea is to have input features that are multiplied by a weight and summed up afterwards. In the simple example below, the network calculates its output by

$$N(\mathbf{x}) = w_{1,1}x_1 + w_{2,1}x_2 \quad (1)$$

The output (neuron) is considered active if the value is larger than 0.

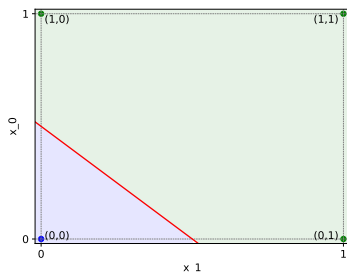
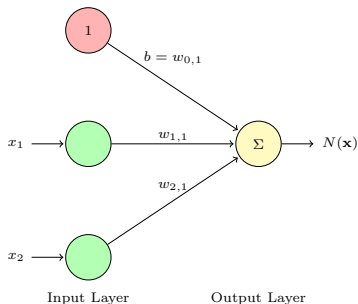


Perceptron: Bias Neuron

Adjusting the weights of the function changes the gradient of the n -dimensional hyperplane (here a straight line for $n = 2$) that, for instance, divides the classes in a classification problem solved with a perceptron.

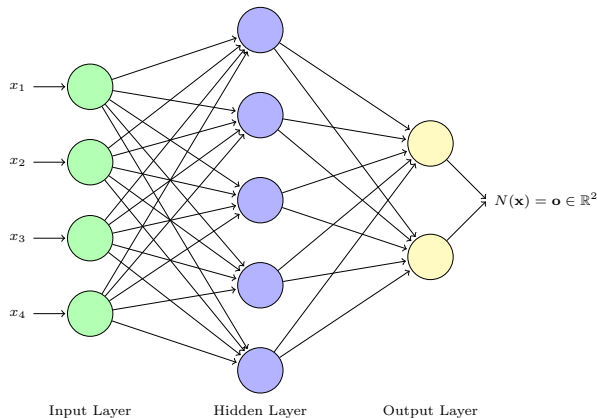
For the simple case of an OR-problem, the classification is seen on the right. All points below the hyperplane – only the case $(0,0)$ – can be classified as `false`, all above as `true`.

To move this hyperplane vertically, we can add a fixed value, independent from any input feature. In neural networks, this is done by a so-called *bias* neuron. Its value is usually always equal to 1, multiplied by a trainable weight like other input neurons. That way, one can train the vertical shift.



Multi-Layer Perceptron: Introduction

The Multi-Layer Perceptron is often considered as the simplest neural network. It uses the concept of a perceptron and combines the results in several layers.



Learning Paradigms and the MNIST Dataset

There are three main learning paradigms in machine learning: supervised-, unsupervised- and reinforcement-learning. Neural networks are usually used in supervised- and reinforcement-learning situations.

In both cases, the network receives feedback based on the result and the expected or desired output. For example, in a classification problem, like digit recognition, the true outcomes are exactly known (the drawn numbers), which makes it a good supervised learning example.

For now (and this course), we will stick with this simple example of digit recognition.

The MNIST dataset is a set of 70,000 small grey-scale pictures with 28×28 pixels, showing a handwritten digit (0-9) each.



For this course, we will divide this dataset into 60,000 training- and 10,000 test-samples.

Multi-Layer Perceptron: Feed Forward

The process to propagate the input features through the network to the output layer is referred to as feed forward.

Each next layer's neuron is calculated by:

$$s_j = \sum_{i=1}^n w_{i,j} x_i + b_j \quad (2)$$

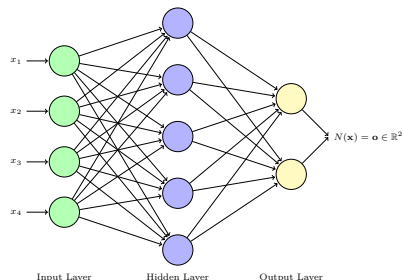
or, as a whole next layer, using linear algebra:

$$\vec{s} = \mathbf{w}^T \vec{x} + \vec{b} \quad (3)$$

Then, a so-called *activation function* is used to break linearity and to avoid collapsing of linear layers into a single layer.

These functions include:

- Leaky Rectified Linear Unit
- Softmax
- TanH
- ...



The final result of the network at the output layer is a function composition of k functions:

$$N(\mathbf{x}) = N^{(k-1)}(N^{(k-2)}(\dots(N^{(0)}(\mathbf{x}))\dots)) \quad (4)$$

Multi-Layer Perceptron: Breaking Linearity

The already mentioned activation functions have to be differentiable functions. Below are some examples that are used quite often:

Rectified Linear Unit (ReLU):

$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Leaky Rectified Linear Unit (LReLU):

$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{otherwise} \end{cases}$$

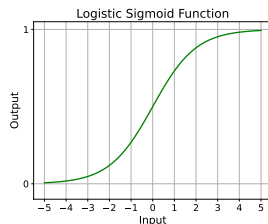
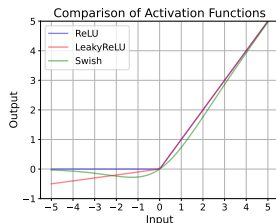
where α is a small positive constant (e.g. 0.01).

Logistic Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Softmax (vector-based function):

$$\text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$



Multi-Layer Perceptron: Loss-/Error-/Cost-Function

After processing the input features to produce an output we now want to let the network "learn" something.

The weights are usually initialized randomly. To make the network learn, one has to define the so-called *loss* that describes how "good" or "bad" an output is.

For the supervised learning problem of digit recognition, the data is labeled with the true shown digit (the true outcome).

For 10 possible digits (classes), we can use 10 output neurons. The neuron corresponding to the predicted digit should be active, all others not. Thus, we create a target value vector $\vec{t}$ containing our target values. The neuron that should be active has a target value of 1, all others 0 — a so-called 1-hot-encoded vector.

Thus, our loss function can use this information to provide our loss some meaningful value. When we consider a classification problem like this, it seems natural to use the difference between our output and our expected/desired target value.

But is it a good idea? We will show: yes!

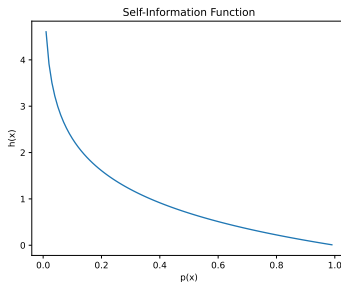
"The basic intuition behind information theory is that learning that an unlikely event has occurred is more informative than learning that a likely event has occurred"

(Deep Learning, Goodfellow et. al., p. 71)

Self-information / Shannon-Information:

$$h(x) = -\log(p(x)) \quad (5)$$

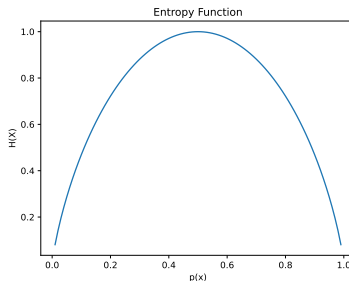
- likely events provide a small amount of information, rare events provide a large amount of information
- limits:
 - $p(x) = 1 \rightarrow h(p(x)) = 0$
 - $p(x) = 0 \rightarrow h(p(x)) = \infty$



An entropy of a random variable X describes the level of uncertainty inherent in the variable's possible outcome. In particular, it quantifies how surprising an event is by providing the average amount of self-information an observer would expect to gain about a random variable, when measuring it. (based on D. S. Jones, Elementary Information Theory, 1979)

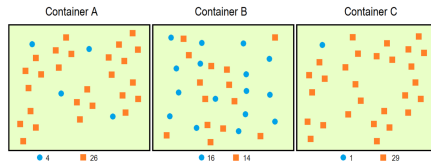
A larger value of $H(X)$ describes higher uncertainty.

$$H(X) = - \sum_x p(x) \log(p(x)) \quad (6)$$



The entropy of a random variable X describes the level of uncertainty inherent in the variables possible outcome.

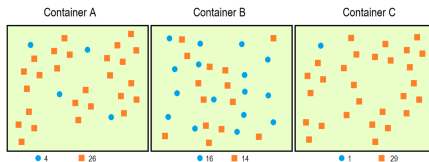
$$H(X) = - \sum_x p(x) \log(p(x))$$



$$H(X) = - \left(\frac{4}{30} \log \left(\frac{4}{30} \right) + \frac{26}{30} \log \left(\frac{26}{30} \right) \right) \approx 0.3927$$

The entropy of a random variable X describes the level of uncertainty inherent in the variables possible outcome.

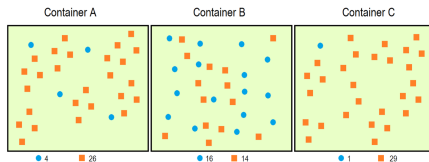
$$H(X) = - \sum_x p(x) \log(p(x))$$



$$H(X) = - \left(\frac{16}{30} \log \left(\frac{16}{30} \right) + \frac{14}{30} \log \left(\frac{14}{30} \right) \right) \approx 0.6909$$

The entropy of a random variable X describes the level of uncertainty inherent in the variables possible outcome.

$$H(X) = - \sum_x p(x) \log(p(x))$$



$$H(X) = - \left(\frac{1}{30} \log \left(\frac{1}{30} \right) + \frac{29}{30} \log \left(\frac{29}{30} \right) \right) \approx 0.1461$$

In our supervised learning problem of digit recognition, there are two probability distributions:

- of the known true outcomes (target values $\vec{t}$)
- of the inferred outcomes by the neural network (output values $\vec{o}$)

Thus, we calculate the cross-entropy between both distributions as a loss function:

$$L_{CE}(\vec{t}, \vec{o}) = - \sum_{c \in C} t_c \log(o_c) \quad (7)$$

This loss describes our network's error. To achieve good results, we should minimize it!

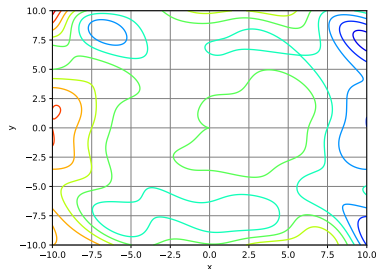
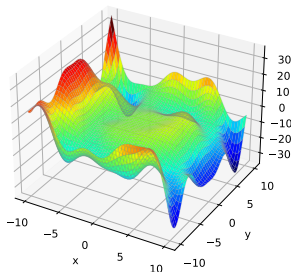
For categorical cross-entropy (where only one of the classes can be true), this can be simplified because only one target value is non-zero (all other classes' target value is 0, indicating the neuron should be inactive):

$$L_{CCE}(t_c, o_c) = -t_c \log(o_c) \quad (8)$$

Finding the Loss-Minimum: Gradient Descent

Remember, we described the network as a function composition. Now, we add another "layer" to that by applying our loss function on the final results:

$$L(N(\mathbf{x})) = L(N(\mathbf{x}) = N^{(k-1)}(N^{(k-2)}(\dots(N^{(0)}(\mathbf{x}))\dots))) \quad (9)$$



(Simplified by using only 3-dimensions)

Gradient Descent (1)

The requirements to calculate the gradients w.r.t. a weight $w_{i,j}$ of $L(N(\mathbf{x})) = L(N(\mathbf{x}) = N^{(k-1)}(N^{(k-2)}(\dots(N^{(0)}(\mathbf{x}))\dots)))$:

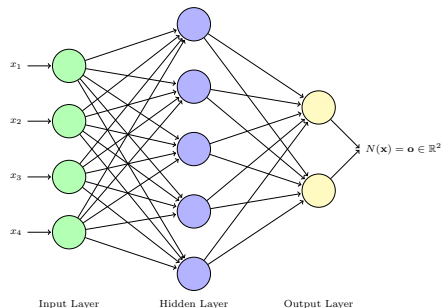
- the functions and operations have to be differentiable
- we have to calculate the derivatives:
 - chain rule (!)
 - product rule
 - quotient rule

We calculated each neuron's value by:

$$o_j = \varphi\left(b_i + \sum_{i=0}^{n-1} w_{i,j}x_i\right) \quad (10)$$

To make it easier, let's substitute the summation by function s_j :

$$s_j = \sum_{i=0}^{n-1} w_{i,j}x_i \quad (11)$$



Gradient Descent (2)

To calculate the gradient for the output neurons' connections, and therefore the gradient of the network's error, we apply chain rule:

$$\frac{\partial L}{\partial w_{i,j}} = \frac{\partial L}{\partial o_j} \frac{\partial o_j}{\partial w_{i,j}} = \frac{\partial L}{\partial o_j} \frac{\partial o_j}{\partial s_j} \frac{\partial s_j}{\partial w_{i,j}} \quad (12)$$

The last factor is trivial, since building the derivative of the sum w.r.t. $w_{i,j}$, all other weights are handled as constant factors and therefore thrown out. The remaining part is $w_{i,j}x_i$, such that:

$$\frac{\partial s_j}{\partial w_{i,j}} = \frac{\partial}{\partial w_{i,j}} \sum_{i=0}^{n-1} w_{i,j}x_i = x_i \quad (13)$$

Let's substitute the two other factors by δ :

$$\delta_j = \frac{\partial L}{\partial o_j} \frac{\partial o_j}{\partial s_j} \quad (14)$$

For output layer neurons, both factors are trivial if loss- and activation-function are known. However, if j is an inner neuron, the derivative is less obvious due to the summation. Then:

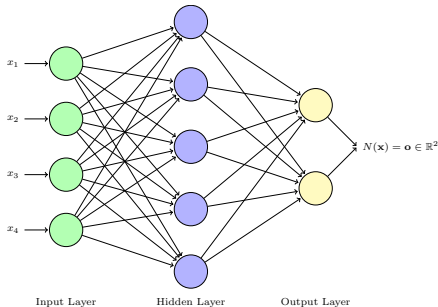
$$\frac{\partial L}{\partial o_j} = \sum_{k \in K} \left(\frac{\partial L}{\partial s_k} \frac{\partial s_k}{\partial o_j} \right) \quad (15)$$

$$= \sum_{k \in K} \left(\frac{\partial L}{\partial o_k} \frac{\partial o_k}{\partial s_k} \frac{\partial s_k}{\partial o_j} \right) \quad (16)$$

$$= \sum_{k \in K} \left(\frac{\partial L}{\partial o_k} \frac{\partial o_k}{\partial s_k} \underbrace{\frac{\partial s_k}{\partial w_{j,k}}}_{\delta_k} w_{j,k} \right) \quad (17)$$

Therefore, for target values $\mathbf{t}$ and output $\mathbf{o}$, with loss function $L(\vec{\mathbf{t}}, \vec{\mathbf{o}})$ and activation function $\varphi(s)$:

$$\delta_j = \frac{\partial L}{\partial o_j} \frac{\partial o_j}{\partial s_j} = \begin{cases} \frac{\partial L(t_j, \varphi(s_j))}{\partial \varphi(s_j)} \frac{d\varphi(s_j)}{ds_j} & \text{if } j \text{ is an output neuron,} \\ \left(\sum_{k \in K} w_{j,k} \delta_k \right) \frac{d\varphi(s_j)}{ds_j} & \text{if } j \text{ is a hidden neuron.} \end{cases} \quad (18)$$



One can then update the weights using $\Delta w_{i,j} = -\eta \frac{\partial L}{\partial w_{i,j}}$, where η is the learning rate $\rightarrow w_{i,j}^{\text{new}} = w_{i,j}^{\text{old}} + \Delta w_{i,j}$.

Softmax-Activation and Cross-Entropy-Loss (1)

For our classification problem, we will use Softmax as the output layer activation function and cross-entropy as a loss function.

Let's try and calculate the derivatives for this case.

The derivative of our cross-entropy loss function can be calculated as:

$$\frac{\partial L_{CE}(\vec{t}, \vec{o})}{\partial s_c} = \frac{\partial \sum_{c=0}^{n-1} t_j \log(o_j)}{\partial s_c} \quad (19)$$

$$= - \sum_{j=0}^{n-1} t_j \frac{\partial \log(o_j)}{\partial s_c} \quad (20)$$

$$= - \sum_{j=0}^{n-1} t_j \frac{\partial \log(o_j)}{\partial o_j} \cdot \frac{\partial o_j}{\partial s_c} \quad (21)$$

$$= - \sum_{j=0}^{n-1} t_j \frac{1}{o_j} \cdot \frac{\partial o_j}{\partial s_c} \quad (22)$$

where $\frac{\partial o_j}{\partial s_c}$ is the derivative of the activated output.

Softmax-Activation and Cross-Entropy-Loss (2)

Activation function for the classification problem: Softmax.

Remember: Softmax (vector-based function):

$$\text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

If o_j is a Softmax-activated output, the function is applied the whole layer. Then, the derivative has to consider the j -th output w.r.t. the c -th input.

Case $c = j$:

$$\frac{\partial o_j}{\partial s_c} = \frac{\partial \frac{e^{s_j}}{\sum_{i=1}^S e^{s_i}}}{\partial s_c} \quad (23)$$

$$= \frac{e^{s_j} (\sum_{i=1}^S e^{s_i}) - e^{s_j} e^{s_c}}{(\sum_{i=1}^S e^{s_i})^2} \quad (24)$$

$$= \frac{e^{s_j}}{\sum_{i=1}^S e^{s_i}} \cdot \frac{(\sum_{i=1}^S e^{s_i}) - e^{s_c}}{\sum_{i=1}^S e^{s_i}} \quad (25)$$

$$= \frac{e^{s_j}}{\sum_{i=1}^S e^{s_i}} \left(\frac{\sum_{i=1}^S e^{s_i}}{\sum_{i=1}^S e^{s_i}} - \frac{e^{s_c}}{\sum_{i=1}^S e^{s_i}} \right) \quad (26)$$

$$= o_j (1 - o_c) \quad (27)$$

Case $c \neq j$:

$$\frac{\partial o_j}{\partial s_c} = \frac{\partial \frac{e^{s_j}}{\sum_{i=1}^S e^{s_i}}}{\partial s_c} \quad (28)$$

$$= \frac{0 - e^{s_c} e^{s_j}}{(\sum_{i=1}^S e^{s_i})^2} \quad (29)$$

$$= - \frac{e^{s_c}}{\sum_{i=1}^S e^{s_i}} \cdot \frac{e^{s_j}}{\sum_{i=1}^S e^{s_i}} \quad (30)$$

$$= -o_c o_j \quad (31)$$

Softmax-Activation and Cross-Entropy-Loss (3)

Now, let us combine everything:

Separating the case $j = c$ from the sum and inserting softmax derivatives for the specific cases:

$$\frac{\partial L_{CE}(\mathbf{t}, \mathbf{o})}{\partial s_c} = \underbrace{-t_j \frac{1}{o_j} \frac{\partial o_j}{\partial s_c}}_{j=c} - \sum_{j \neq c} t_j \frac{1}{o_j} \cdot \frac{\partial o_j}{\partial s_c} \quad (34)$$

CE-Loss derivative

$$\frac{\partial L_{CE}(\vec{\mathbf{t}}, \vec{\mathbf{o}})}{\partial s_c} = - \sum_{j=0}^{n-1} t_j \frac{1}{o_j} \cdot \frac{\partial o_j}{\partial s_c} \quad (32)$$

Softmax derivative

$$f(x) = \begin{cases} o_j(1 - o_c) & \text{if } c = j \\ -o_j o_c & \text{otherwise} \end{cases} \quad (33)$$

$$= -t_c \frac{1}{o_c} o_c (1 - o_c) - \sum_{j \neq c} t_j \frac{1}{o_j} \cdot \frac{\partial o_j}{\partial s_c} \quad (35)$$

$$= -t_c (1 - o_c) - \sum_{j \neq c} t_j \frac{1}{o_j} \cdot (-o_c o_j) \quad (36)$$

$$= -t_c (1 - o_c) + \sum_{j \neq c} t_j o_c \quad (37)$$

$$= -t_c + t_c o_c + \sum_{j \neq c} t_j o_c \quad (38)$$

$$= -t_c + o_c \left(t_c + \sum_{j \neq c} t_j \right) \quad (39)$$

Since $\sum_j t_j = 1$ by definition because $\vec{\mathbf{t}}$ is a 1-hot encoded vector, $t_c + \sum_{j \neq c} t_j = 1$, and therefore, $o_c \cdot 1 - t_c = o_c - t_c$, which is the difference between the predicted output value and the target value for neuron c — our first guess.

There are many topics to cover for a deep dive into machine learning and neural networks and it is a really interesting topic in general, that also arises in high-performance computing.

Since machine learning is not the focus of this lecture, we will not focus on them here. Nevertheless, you might have heard about them when working in this area.

These include:

- Number of epochs, batch size, learning rate and other hyperparameters
- Different datasets for training, validation, and testing
- Underfitting / Overfitting of models
- Parameter optimization techniques other than "vanilla" gradient descent
- Regularization methods
- and many others...

Please have a look into our script for more information on these.